UNIT - IV

Backtracking : The general method – 8 queens problem – sum of subsets – graph coloring – Hamiltonian cycle – Knapsack problem.

4.1. GENERAL METHOD:

- It is one of the most general algorithm design techniques.
- Many problems which deal with searching for a set of solutions or for a optimal solution satisfying some constraints can be solved using the backtracking formulation.
- To apply backtracking method, the desired solution must be expressed as an n-tuple ($x_1,\ldots x_n$) where x_i is chosen from some finite set Si and m is the size of set Si.
- The problem is to find a vector, which maximizes or minimizes a criterion function $P(x_1,\ldots x_n)$.
- The major advantage of this method is, once we know that a partial vector ($x_1,\ldots x_i$) will not lead to an optimal solution that (m_{i+1}……..m_n) possible test vectors may be ignored entirely.
- Brute Force Method:

  It forms all the n-tuples ,evaluate each one with P & save these which yield the optimum..
- BackTracking Algorithm:

  It has ability to yield the same answer with far fewer than m trials.
  - Basic Idea:

    1.Bulil up the solution vector one component at a time.

    2.Use modified criterion functions P(x1…. x_i) to test whether the vector being formed has any chance of success.
- Advantages:

  If it is realized that the partial vector (x1,x2…. x_i) can in no way lead to an optimal solution,then m_{i+1}…..m_n possible test vectors can be ignored entirely.
- Backtracking requires that all the solutions should satisfy a complex set of constraints.
- These constraints are classified as:

  1. Explicit constraints. 2. Implicit constraints.

1) Explicit constraints:

- Explicit constraints are rules that restrict each x_i to take values only from a given set. Some examples are,

  $x_i \geq 0$ or Si ={all non-negative real nos.}

  $x_i = 0$ or 1 or Si={0,1}.

  $l_i \leq x_i \leq u_i$ or Si={a : $l_i \leq a \leq u_i$ }
- Explicit constraints depend on the particular instance I of the problem being solved.

- All tuples that satisfy the explicit constraint define a possible solution space for I.

2) Implicit constraints:

- The implicit constraints are rules that determine which of the tuples in the solution space of I can actually satisfy the criterion function.
- Describe the way in which the x_i must relate to each other.

Algorithm:

```
Algorithm IBacktracking (n)
// This schema describes the backtracking process.
//All solutions are generated in x[1:n]

//and printed as soon as they are determined.
{
   k:=1;
   while (k ≠ 0) do
   {
     if (there remains an untried x[k] ∈ T (x[1],[2],…..x[k-1]) and
                                    Bk (x[1],…..x[k]) is true ) then
     {
         if(x[1],……x[k] ) is the path to the answer node) then
                 write(x[1:k]);
                 k=k+1;    //consider the next step.
     }
     else
         k=k-1;            //consider backtracking to the previous set.
   }
 }
```

- All solutions are generated in x[1:n] and printed as soon as they are determined.
- T( ) yields the set of all possible values that can be placed as the first component x1 of the solution vector.
- T(x[1]…..x[k-1]) is all possible values of x[k] gives that x[1],……..x[k-1] have already been chosen.
- B_k(x[1]……x[k]) is a boundary function which determines the elements of x[k] which satisfies the implicit constraint.
- If bounding function is $B_1(x_1)$ is true, then the component x1 will take on those values.
- The variable k is incremented and the solution vector is grown until solution is found or no untried value of x_k remains.
- If the k is decremented, the algorithm must resume the generation of possible elements for the kth position that had not yet been tried.
- Certain problems which are solved using backtracking method are,

1.Sum of subsets.
2.Graph coloring.
3.Hamiltonian cycle.
4.N-Queens problem.

4.2. N-QUEENS PROBLEM:

MVIT-CSE

The 8 queens problem is to place n-queens in a n*n matrix in such a way that no two queens attack each i.e: no two queens should be in the same row, column and diagonal.

Problem:

- We have 8 queens and 8*8 chessboard . i.e: rows and 8 columns.
- Each queen should be placed on the different row.
- Assume Queen i is placed on row i.
- The solution vector x ($x_1 \ldots x_n$) represents a solution in which x_i is the column of the ith row where i th queen is placed.
- Here all solutions can be represented as 8 tuples($x_1 \ldots x_8$) where x_i is the column of ith row on which Queen i is placed.
- **Explicit Constraints:**
  - $S_i = \{1,2,3,4,5,6,7,8\}, 1 \le i \le 8$

  $\therefore$ Solution space consist of 8^8 tuples.
- **Implicit Constraints:**
  - No 2 xi's can be the same.

  i.e)All queens must be on different columns and diagonals
- From the implicit constraints,

  All solutions are permutations of the 8 tuple.It reduces the size of the solution space from 8^8 tuples to 8! tuples.

Solution:

- Chessboard squares are being numbered as indices of 2D array a[1:n,1:n].
- First we have to check no two queens are in same row.
- Second we have to check no two queens are in same column.
- The function, which is used to check these two conditions, is [i,x(j)] which gives position of the ith queen, where i represents the row and x (j) represents the column position.
- Third we have to check no two queens are in same diagonal.
- Consider two dimensional array a[1:n,1:n] in which we observe that every element on the same diagonal that runs from upper left to lower right has the same row-column value.

  Ex:

  a[3,1],a[4,2],a[5,3],a[6,4],a[7,5],a[8,4]

  All these squares have same row – column value .i.e)2

Also, every element on the same diagonal that runs from upper right to lower left has the same row + column value.

Ex: a[2,6], a[3,5],a[4,4],a[5,3],a[6,2],a[7,1]

All these squares have same row + column value .i.e)8

- Suppose two queens are in same position (i,j) and (k,l) then two queens lie on the same diagonal , if and only if |j-l|=|i-k|.

Example:

For 4 queens Problem,there are only 2 Solutions.They are,

MVIT-CSE

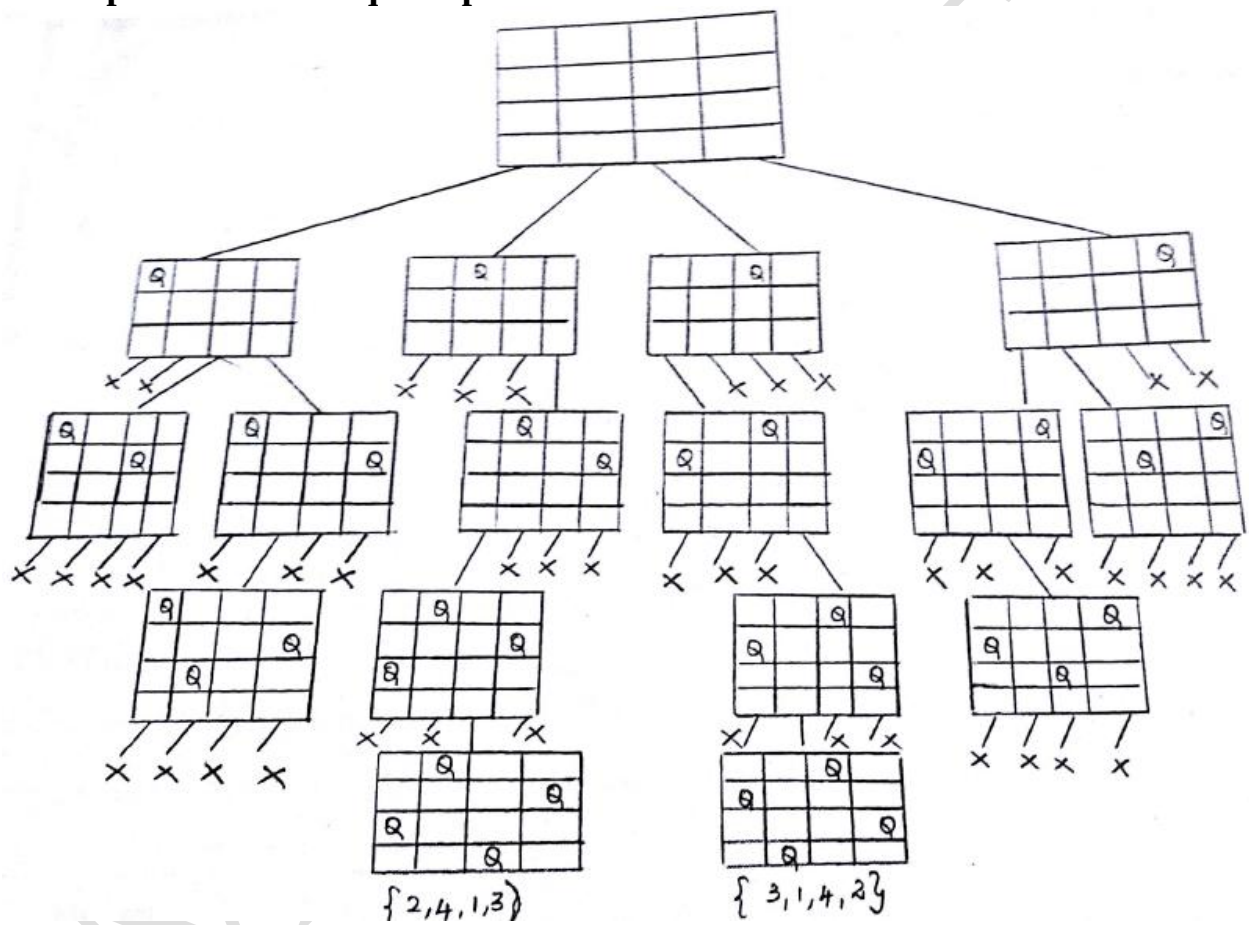
	Q		
			Q
Q			
		Q	

Solution1
(2 4 1 3)

		Q	
Q			
			Q
	Q		

Solution 2
(3 1 4 2)

State space Tree for 4 queen problem



Algorithm:

Algorithm Nqueens (k,n)

```
//Using backtracking it prints all possible positions of n queens in 'n*n' chess board.
// So that they are non-tracking.
{
    for i=1 to n do
    {
        if place (k,i) then
        {
            x [k]=i;
                if (k=n) then
                        write (x [1:n]);
```

```
                else
                        Nqueens(k+1,n)    ;

        }
    }
}
```

Algorithm place (k,i)

```
//return true if a queen cam be placed in k th row and I th column.
// otherwise it returns false.
//x[] is a global array whose first k-1 values have been set.
//Abs( r) returns the absolute value of r.
{
        for j=1 to k-1 do
                if ((x [j]=i)           //two in same column.
                        or (abs (x [j]-i)=Abs (j-k))) then
                        return false;
        return true;
}
```

Algorithm Nqueens (k,n):

- Gives the precise solution to the n queens problem.
- Array x[] is global.
- Algorithm is invoked by Nqueens(1,n).

Algorithm place (k,i):

- This function returns a Boolean value that is true if the kth queen can be placed in column i.
- It checks whether I is distinct from all previous values x[1]……x[k-1].
- It checks whether there is no queen on the same diagonal.
- Time Complexity is *O*(k-1)

Explanation of Algorithm:(Steps to generate the Solution)

- Initialize x array to zero and start by placing the first queen in k=1 in the first row.
- To find the column position start from value 1 to n, where 'n' is the no. of columns or no. of queens.
- If k=1 then x (k)=1.so (k,x(k)) will give the position of the k th queen. Here we have to check whether there is any queen in the same column or diagonal.
- For this, consider the previous position, which had already been found out. Check whether
- x (i)=x(k) for column |x(i)-x(k)|=(i-k) for the same diagonal.
- If any one of the condition is true, then return false indicating that k th queen can't be placed in position x(k).
- For not possible condition increment x(k) value by one and proceed until the position is found.
- If the position x(k)≤ n and k=n then the solution is generated completely.
- If k<n,then increment the 'k' value and find position of the next queen.
- If the position x(k)>n then kth queen can't be placed as the size of the matrix is 'n*n'.
- So decrement the 'k' value by one i.e. we have to backtrack and after the position of the previous queen.

MVIT-CSE

4.3. SUM OF SUBSETS

- We are given 'n' positive numbers called weights and we have to find all combinations of these numbers whose sum is M. This is called as sum of subsets problem.
- If we consider backtracking procedure using fixed tuple strategy , the elements x(i) of the solution vector is either 1 or 0 depending on if the weight w(i) is included or not.
- If the state space tree of the solution, for a node at level i, the left child corresponds to x(i)=1 and right to x(i)=0.

Example:

- Given n=6,M=30 and w(1…6)=(5,10,12,13,15,18).We have to generate all possible combinations of subsets whose sum is equal to the given value M=30.
- In state space tree of the solution the rectangular node lists the values of s, k, r where ,

  s is the sum of subsets,
  k is the iteration
  r is the sum of elements after 'k' in the original set.

GENERATION OF STATE SPACE TREE:

- Maintain an array x to represent all elements in the set.
- The value of x_i indicates whether the weight wi is included or not.
- Sum is initialized to 0. i.e: s=0.
- We have to check starting from the first node.
- Assign x(k) = 1.
- If s+x(k)=M , then we print the subset. B'coz the sum is the required output.
- If the above condition is not satisfied then we have to check s+x(k)+w(k+1)<=M. If so we have to generate the left sub tree. It means w(t) can be included so the sum will be incremented and we have to check for the next k.
- After generating the left sub tree we have to generate the right sub tree.For this we have to check s+w(k+1)<=M. B'coz w(k) is omitted and w(k+1) has to be selected.
- Repeat the process and find all the possible combinations of the subset.
-

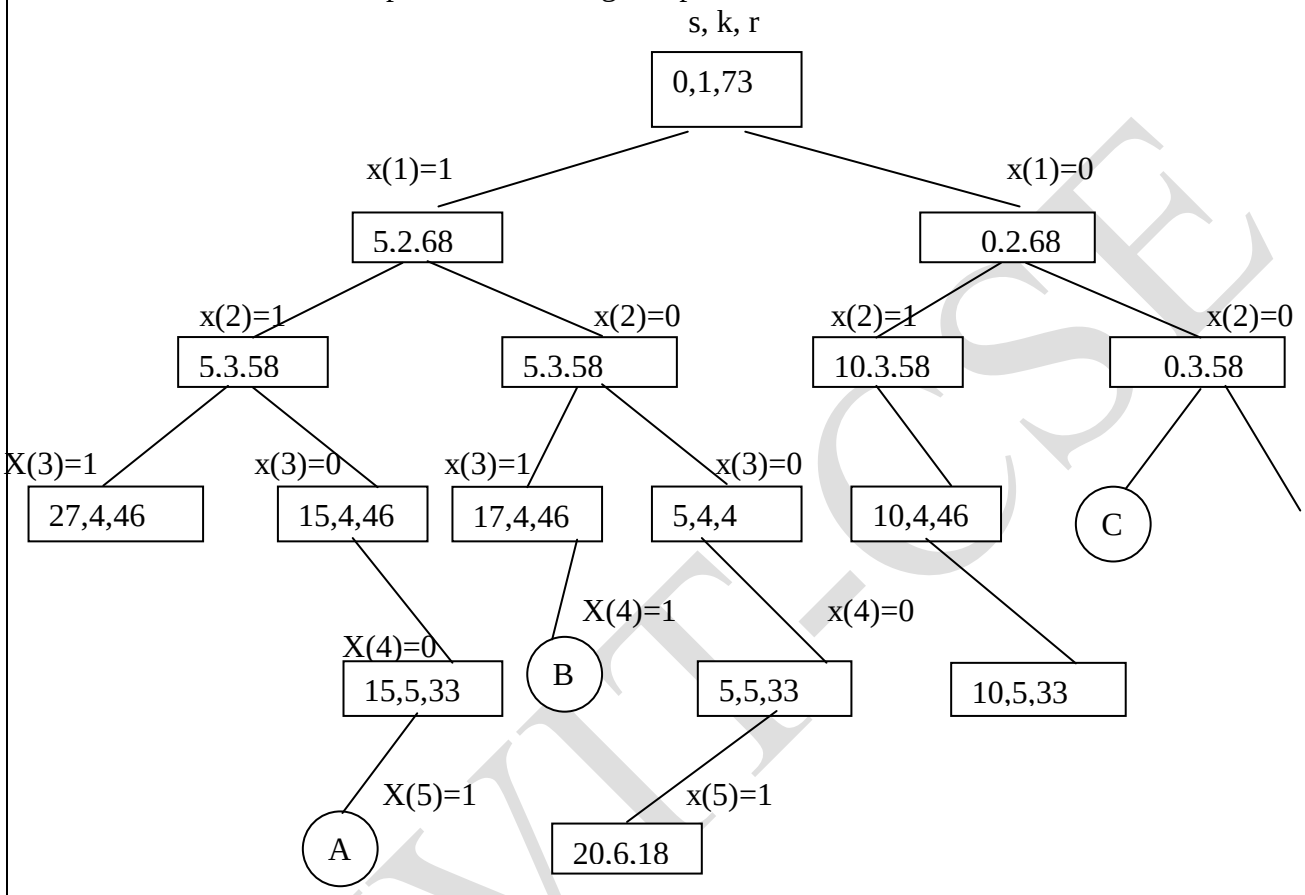
Recursive Backtracking Algorithm for Sum of Subsets Problem:

```
Algorithm SumOfSubset(s,k,r)
{
//Find all subsets of w[1:n].
//generate the left child.note s+w(k)<=M since Bk-1 is true.
x{k]=1;
if (s+w[k]=M) then
write(x[1:k]);
 // there is no recursive call here as w[j]>0,1<=j<=n.
else
if (s+s[k]+s[k+1]<=M) then
SumOfSubset(s+w[k], k+1,r- w[k]);
//generate right child and evaluate Bk.
if ((s+ r- s[k]>=m)and(s+ s[k+1]<=M)) then
```

MVJT-CSE

```
{
 x{k]=0;
                SumOfSubset (s, k+1, r-w[k]);
}
}
```

- The state space tree for the given problem is,



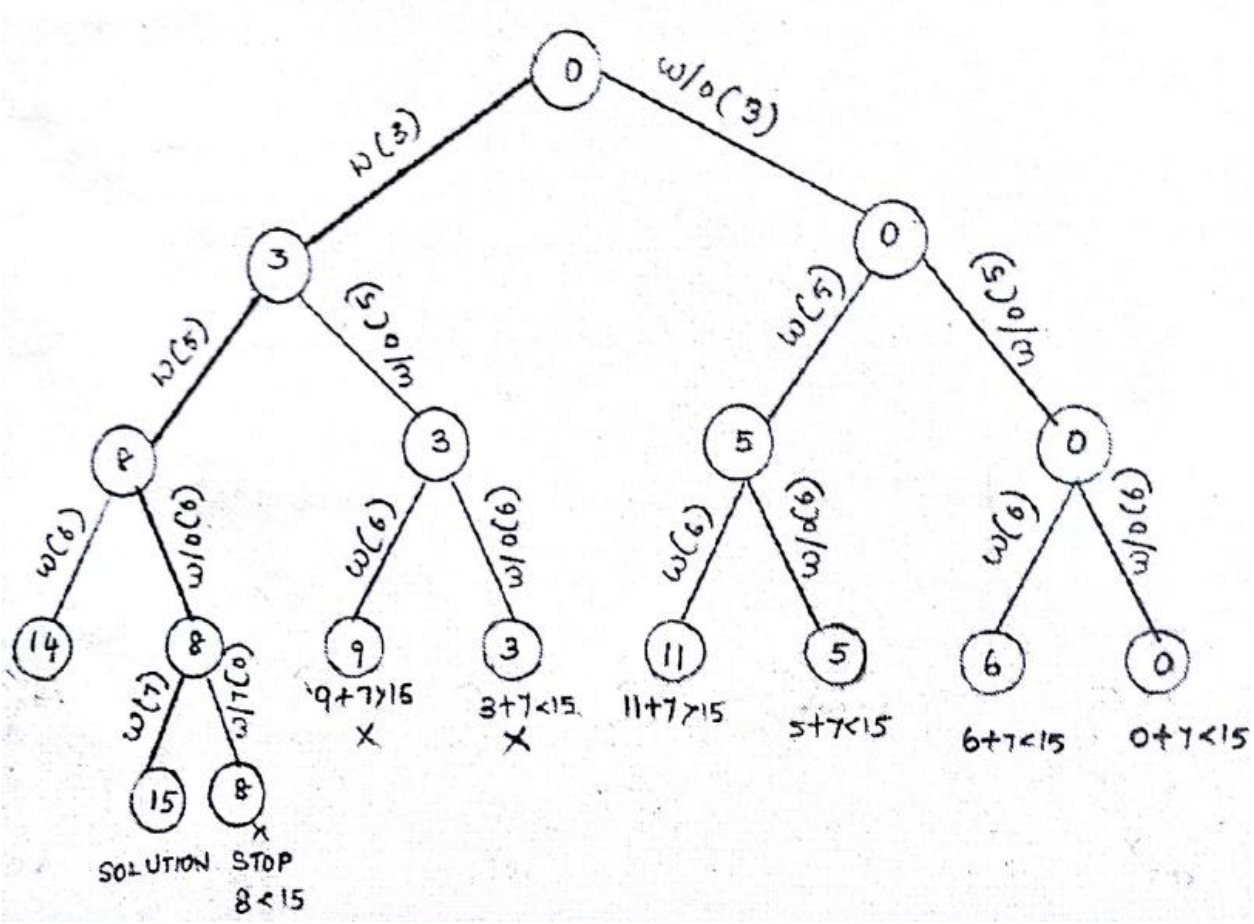
Solution **A** -> 1 1 0 0 1 0
Solution **B** -> 1 0 1 1 0 0
Solution **C** -> 0 0 1 0 0 1

- In the state space tree, edges from level 'i' nodes to 'i+1' nodes are labeled with the values of x_i, which is either 0 or 1.
- The left sub tree of the root defines all subsets containing w_i.
- The right subtree of the root defines all subsets, which does not include w_i.
- Full state space tree contains 26-1 =63 nodes.(n=6) from which calls to be made.(No calls from leaf nodes)

EXAMPLE PROBLEMS:

MVIT-CSE

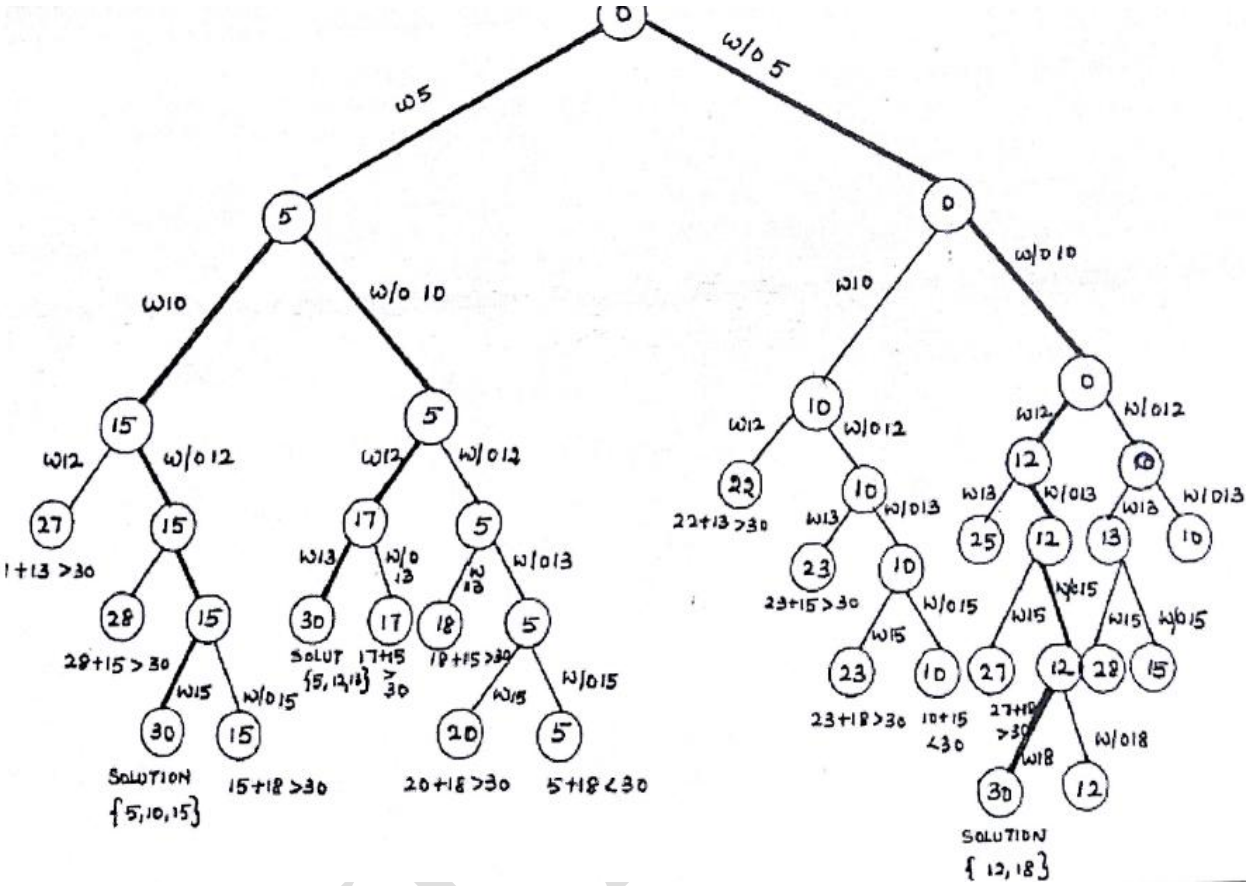
- We are given ‘n’ positive numbers called weights and we have to find all combinations of these numbers whose sum is M. This is called as sum of subsets problem.
- If we consider backtracking procedure using fixed tuple strategy , the elements x(i) of the solution vector is either 1 or 0 depending on if the weight w(i) is included or not.
- If the state space tree of the solution, for a node at level i, the left child corresponds to x(i)=1 and right to x(i)=0.
- In the state space tree of the solution for a node at level I left child corresponds to X(i)=1 and right child X(i)=0.
- **Example:**
- Given n=4, M=12 and W(2,4,6,8) generate the possible sum of subset whose sum is equal to M. SOLUTION: 1110-→12 & 0101 -→12.
- Given n=4, M=15 and W(3,5,6,7) generate the possible sum of subset whose sum is equal to M. SOLUTION: 1101



- In the state space tree edges from level I nodes to level i+1 nodes are labeled with values of xi which is either 0 or 1.
- The left sub tree of the root defines all subset containing Wi.
- The right sub tree defines all subsets which doesnot include Wi.

Problem 2:

- **Given n=6, M=30 and W(5,10,12,13,15,18) generate the possible sum of subset whose sum is equal to M**



Problems for practiceProblems for practiceProblems for practice

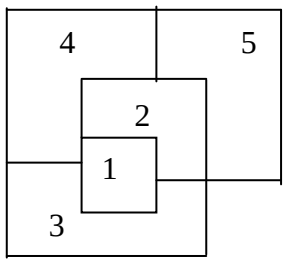
1. w={5,10,12,13,15,18}, M=30, n=6
- Solution : 110010 & 101100 & 001001

2. w={2,3,11,13,24,7,26}, M=31, n=7

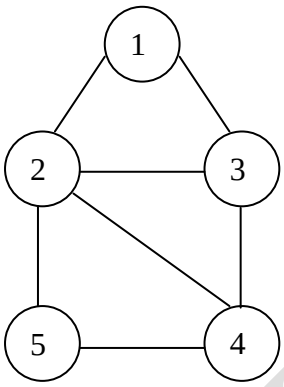
4.4. GRAPH COLORING:

- Let ‘G’ be a graph and ‘m’ be a given positive integer. If the nodes of ‘G’ can be colored in such a way that no two adjacent nodes have the same color. Yet only ‘M’ colors are used. So it’s called M-color ability decision problem.
- The graph G can be colored using the smallest integer ‘m’. This integer is referred to as chromatic number of the graph.
- If ‘d’ is the degree of the given graph, then it can be colored with d+1 colors.
- A graph is said to be planar iff it can be drawn on plane in such a way that no two edges cross each other.
- Suppose we are given a map then, we have to convert it into planar. Consider each and every region as a node. If two regions are adjacent then the corresponding nodes are joined by an edge.

Consider a map with five regions and its graph.



1 is adjacent to 2, 3, 4.
2 is adjacent to 1, 3, 4, 5
3 is adjacent to 1, 2, 4
4 is adjacent to 1, 2, 3, 5
5 is adjacent to 2, 4

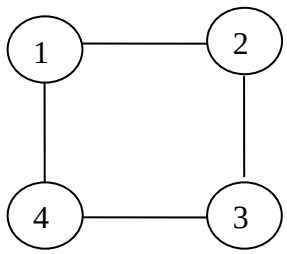


Steps to color the Graph:

- First create the adjacency matrix (1:n,1:n) for a graph.
  if there is an edge between i,j then C(i,j) = 1
  otherwise C(i,j) =0.
- The Colors will be represented by the integers 1,2,…..m.
- The solutions will be stored in the array x(1),x(2),……..,x(n) where x(index) is the color & index is the node.
- He formula is used to set the color is,
  - x(k) = (x(k)+1) % (m+1)
- First one chromatic number is assigned ,after assigning a number for 'k' node,we have to check whether the adjacent nodes has got the same values if so then we have to assign the next value.
- Repeat the procedure until all possible combinations of colors are found.
- The function which is used to check the adjacent nodes and same color is,
  - If((Graph (k,j) == 1) and x(k) = x(j))

MVIT-CSE

Example:



n= 4
M= 3

Adjacency Matrix:

$$\begin{vmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{vmatrix}$$

- Problem is to color the given graph of 4 nodes using 3 colors.
- Node 1 can take the given graph of 4 nodes using 3 colors.
- The state space tree is drawn. It is a tree of degree m with height n+1.
- The state space tree will give all possible colors in that ,the numbers which are inside the circles are nodes ,and the branch with a number is the colors of the nodes.
- Each node at level i has m children corresponding to the m possible assignments to xi,1≤i≤n.
- Nodes at level n+1 are leaf nodes.

Algorithm:Finding all m-colorings of a graph

Algorithm mColoring(k)

```
// This algorithm was formed using recursive backtracking scheme.
//The graph is represented by its Boolean adjacency matrix G[1:n,1:n].
//All assignments of 1,2,…….,m to the vertices of the graph such that
// adjacenct vertices are assigned distinct integers are printed.
//'k' is the index of the next vertex to color.
{
      repeat
     {
        // Generate all legal assignment for x[k].
        NextValue(k);    // Assign to x[k] a legal color.
        if (x[k]=0) then return;          // No new color possible.
        if (k=n) then  // Atmost 'm' colors have been used to color the 'n' vertices
                        write(x[1:n]);
        else mColoring(k+1);
     }until(false);
}
```

MVITCSE

Algorithm:Generating a next color
Algorithm NextValue(k)

```
//x[1],……x[k-1] have been assigned integer values in the range[1,m] such that
//adjacent values have distinct integers.
//A value for x[k] is determined in the range[0,m].
//x[k] is assigned the next highest numbered color while maintaining distinctness
//from the adjacent vertices of vertex k.
//If no such color exists, then x[k] is 0.
{
    repeat
      {
         x[k] = (x[k]+1)mod(m+1);    // next highest color.
         if(x[k]=0) then return;          //All colors have been used.
         for j=1 to n do
         {
            // Check if this color is distinct from adjacent colors.
            if((G[k,j] ≠ 0)and(x[k] =x[j]))
            // if (k,j) is an edge and if adjacent vertices have the same color.
              then break;
         }
         if(j=n+1) then return;     //new color found.
      } until(false);    //otherwise try to find another color.
}
```

Algorithm mColoring:

- Assign the graph to adjacency matrix.
- Assign array x[] as 0.
- Initially it invoked ny mColoring(1).
- mColoring repeatedly picks an element from the set of possibilities and assign it to x(k).

Algorithm NextValue:

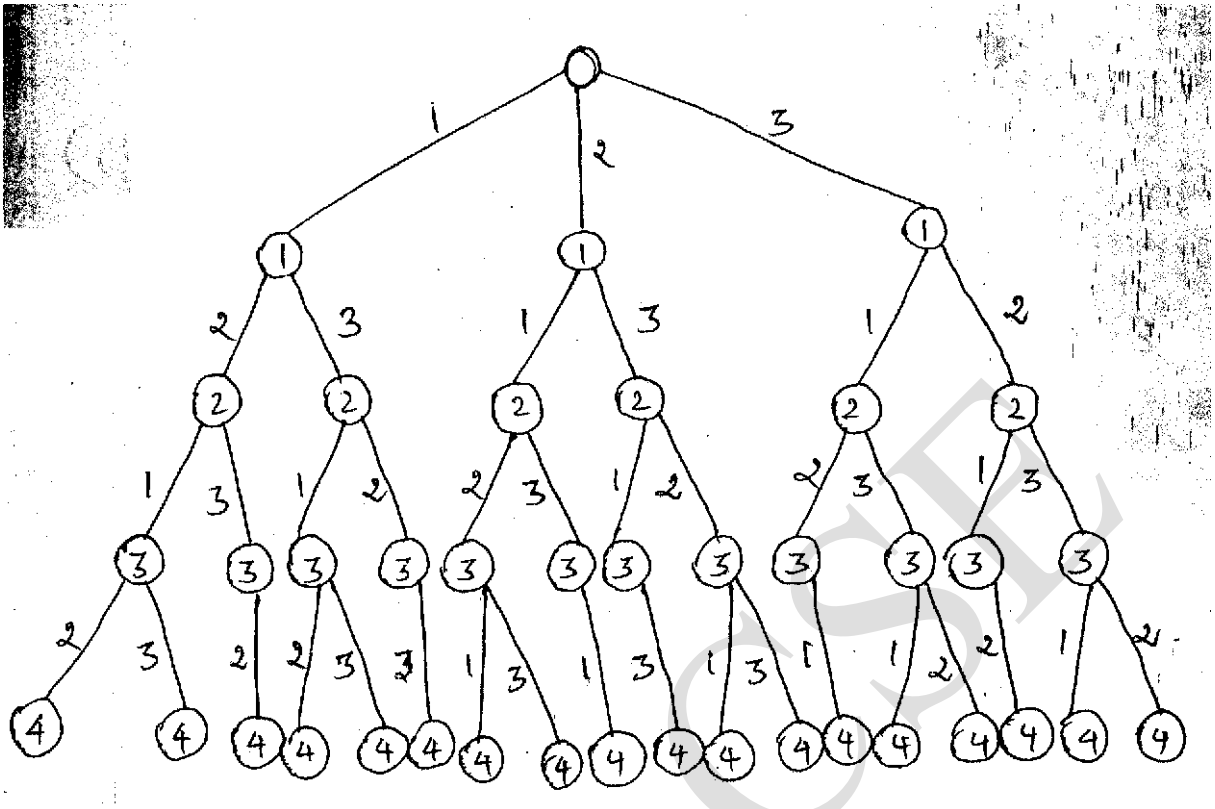
- It produces the possible colors of x(k) after x(1) thru' x(k-1) have been defined.

Analysis:

- Computing time of mColoring (determined from state space tree) is $\sum_{i=0}^{n-1} m^i$
- Time spent by NextValue to determine the children is $O(mn)$
- Total time is $O(m^n n)$.

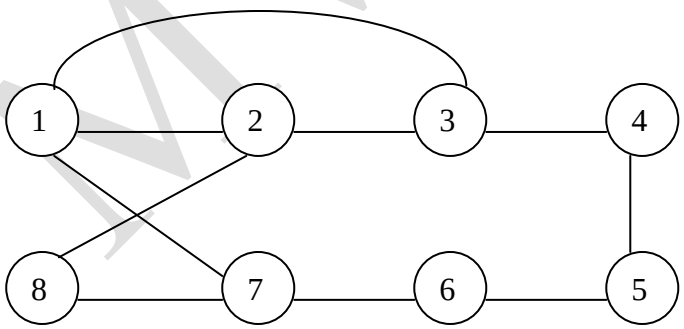
State Space Tree:

MVIT-CSE



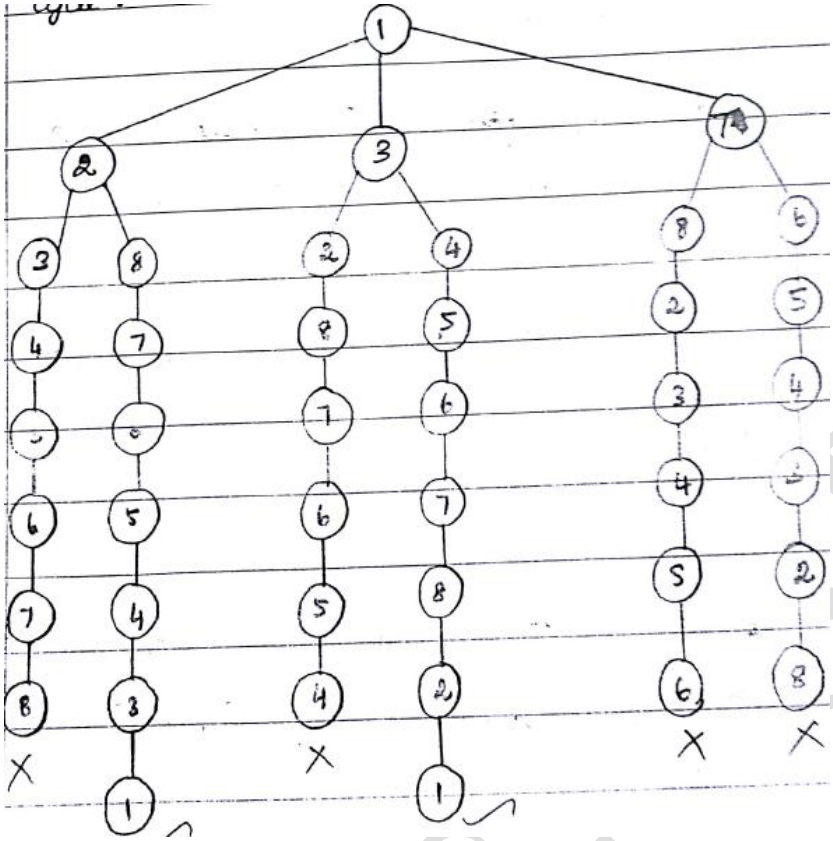
4.5. HAMILTONIAN CYCLES

- Let G=(V,E) be a connected graph with 'n' vertices.
- A Hamiltonian Cycle is a round trip path along 'n' edges of G that visits every vertex once and returns to its starting position.
- If the Hamiltonian cycle begins at some vertex V_1 belongs to G and the vertices of G are visited in the order of $V_1, V_2 \ldots\ldots V_{n+1}$, then the edges (V_i, V_{i+1}) are in E, $1<=i<=n$ and the V_i are distinct except V_1 and V_{n+1} which are equal.
- Consider an example graph G1.:

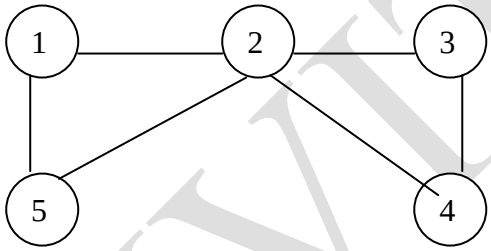


The graph G1 has following Hamiltonian cycles:
Cycle1:1,3,4,5,6,7,8,2,1
Cycle2:1,2,8,7,6,5,4,3,1.

MVIT-CSE



- Consider an another example graph G2:



The Graph G2 has no Hamiltonian cycles.

- The backtracking algorithm helps to find Hamiltonian cycle for any type of graph.(whether directed or undirected)

Procedure:

- Define a solution vector $x(x_i \ldots\ldots x_n)$ where xi represents the ith visited vertex of the proposed cycle.
- Create a cost adjacency matrix for the given graph.
- The solution array initialized to all zeros except x(1)=1,B'coz the cycle should start at vertex '1'.To avoid printing the same cycle n times,we require that x_1=1.
- Now we have to find the second vertex to be visited in the cycle.
- The vertex from 1 to n are included in the cycle one by one by checking 2 conditions,
    - There should be a path from previous visited vertex to current vertex.

- The current vertex must be distinct and shouldn't have been visited earlier.

- When these two conditions are satisfied the current vertex is included in the cycle, else the next vertex is tried.
- When the nth vertex is visited we have to check, is there any path from nth vertex to first vertex. if no path, the go back one step and after the previous visited node.
- Repeat the above steps to generate possible Hamiltonian cycle.

Algorithm for Finding all Hamiltonian cycles:

```
Algorithm Hamiltonian (k)
//This algorithm uses the recursive formulation of backtracking
//to find all the hamiltonian cycles of a graph.
//The graph is stored as an adjacency matrix G[1:n,1:n].
//All cycles begin at node 1.
{
        repeat
        {
                //generate values for x[k].
                Nextvalue(k);    //Assign a legal next value to x[k].
                if (x (k)=0) then return;
                if (k=n) then
                        write (x[1:n]);
          else
                        Hamiltonian (k+1);
        }until(false);
}
```

- This algorithm is started by first initializing the adjacency matrix G[1;n,1:n].
- Then set x[2:n] to zero and x[1] to 1.
- Then execute Hamiltonian(2).

Algorithm for generating a next vertex:

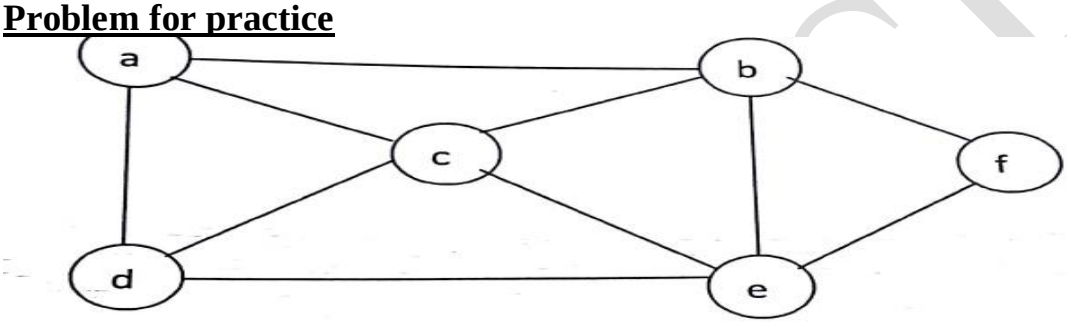
```
Algorithm Nextvalue (k)
         //x[1:k - 1] is a path of k - 1 distinct vertices.
        //if x[k]=0 then no vertex has as yet been assigned to x[k].
        //After execution,x[k] is assigned to the next highest numbered
        //vertex which does not already appear in x[1: k - 1] and
        //is connected by an edge to x[k - 1].Otherwise x[k]=0.
        //if k=n,then in addition x[k] is connected to x[1].
{
 repeat
        {
                  x[k]:=(x [k]+1) mod (n+1); //next vertex.
                  if (x [k] = 0) then return;
                if (G [x [k - 1], x[k]] ≠ 0)then
                {
```


MVIT-CSE

```
                    //Is there an edge?
                    for j:=1 to k - 1 do
                            if (x[j]=x[k]) then break;
                                    // Check for distinctness.
                    if (j=k) then          //if true, then the vertex is distinct.
                            if ((k<n) or ((k=n) and G [x [n], x [1]] ≠ 0))
                                    then return;
            }
    }until (false);
}
```

- This algorithm determines the possible next vertex for the proposed cycle.

Problem for practice



	a	b	c	d	e	f
a	0	1	1	1	0	0
b	1	0	1	0	1	1
c	1	1	0	1	1	0
d	1	0	1	0	1	0
e	0	1	1	1	0	1
f	0	1	0	0	1	0

4.6. KANPSACK PROBLEM USING BACKTRACKING

- The problem is similar to the zero-one (0/1) knapsack optimization problem is dynamic programming algorithm.

- We are given 'n' positive weights Wi and 'n' positive profits Pi, and a positive number 'm' that is the knapsack capacity, the is problem calls for choosing a subset of the weights such that,

$$\sum_{1\le i\le n} WiXi \le m \text{ and } \sum_{1\le i\le n} PiXi \text{ is Maximized.}$$

Xi →Constitute Zero-one valued Vector.

- The Solution space is the same as that for the sum of subset's problem.

- Bounding functions are needed to help kill some live nodes without expanding them. A good bounding function for this problem is obtained by using an upper bound on the value of the best feasible solution obtainable by expanding the given live node.

- The profits and weights are assigned in descending order depend upon the ratio.

  (i.e.) Pi/Wi $\geq$ P(I+1) / W(I+1)

Solution:

- After assigning the profit and weights ,we have to take the first object weights and check if the first weight is less than or equal to the capacity, if so then we include that object (i.e.) the unit is 1.(i.e.) K→ 1.

- Then We are going to the next object, if the object weight is exceeded that object does not fit. So unit of that object is '0'.(i.e.) K=0.
- Then We are going to the bounding function ,this function determines an upper bound on the best solution obtainable at level K+1.

- Repeat the process until we reach the optimal solution.

PROBLEMS

- **Solve the Following KANPSACK problem using Backtracking (SST)** m=15, (w1,w2,w3)=(2,3,4) Pi={1,2,5} n=3.
- **Solution: Pt=6 & Wt=6 & 1101**
- Solve the Following KANPSACK problem using Backtracking (SST) m=16, (w1,w2,w3,w4)=(2,5,10,5) Pi={40,30,50,10} n=4.
- **Solution : wt =15 & Pt=80 wt =15 pt =60**

Algorithm:

Algorithm Bknap(k,cp,cw)

// 'm' is the size of the knapsack; 'n' → no.of weights & profits. W[]&P[] are the //weights & weights. P[I]/W[I] $\geq$ P[I+1]/W[I+1].
//fw→Final weights of knapsack.
//fp→ final max.profit.
//x[k] = 0 if W[k] is not the knapsack,else X[k]=1.

```
{
    // Generate left child.
     If((W+W[k] ≤m) then
```

```
        {
            Y[k] =1;
             If(k<n) then Bnap(k+1,cp+P[k],Cw +W[k])
               If((Cp + p[w] > fp) and (k=n)) then

                {
                  fp = cp + P[k];
                  fw = Cw+W[k];
                 for j=1 to k do X[j] = Y[j];
               }
    }

  if(Bound(cp,cw,k) ≥ fp) then
  {
       y[k] = 0;
     if(k<n) then Bnap (K+1,cp,cw);
   if((cp>fp) and (k=n)) then
      {
          fp = cp;
           fw = cw;
             for j=1 to k do X[j] = Y[j];
       }
    }
}
```

Algorithm for bounding function:

```
Algorithm Bound(cp,cw,k)
// cp→ current profit total.
//cw→ current weight total.
//k→the index of the last removed item.
//m→the knapsack size.

{
    b=cp;
    c=cw;
    for I =- k+1 to n do
  {
        c= c+w[I];
       if (c<m) then b=b+p[I];
           else return b+ (1-(c-m)/W[I]) * P[I];
}
return b;
}
```

Example:

M= 6 Wi = 2,3,4 4 2 2

MVIT-CSE

N= 3 Pi = 1,2,5 Pi/Wi (i.e.) 5 2 1

Xi = 1 0 1
The maximum weight is 6

The Maximum profit is (1*5) + (0*2) + (1*1)
→ 5+1
→ 6.

Fp = (-1)

- $1 \leq 3$ & $0+4 \leq 6$
  cw = 4,cp = 5,y(1) =1
  k = k+2

- $2 \leq 3$ but 7>6
  so y(2) = 0

- So bound(5,4,2,6)

B=5
C=4
I=3 to 3
C=6
$6 \neq 6$
So return 5+(1-(6-6))/(2*1)

- 5.5 is not less than fp.
  So, k=k+1 (i.e.) 3.

  3=3 & $4+2 \leq 6$

  cw= 6,cp = 6, y(3)=1.
  K=4.

- If 4> 3 then
  Fp =6,fw=6,k=3 ,x(1) 1 0 1
  The solution Xi → 1 0 1

  Profit → 6
  Weight →6.

MVIT-CSE